WEEK 05

Conversion and exceptions

Python foundations

Conversion and exceptions

External text, useful errors and an audit trail

Convert text

Numeric text and numbers behave differently.

```
print("4" * 2)       # 44
print(int("4") * 2)  # 8
```

Validate the value

Successful conversion does not imply an allowed range.

```
count = int("0")
print(count > 0)  # False
```

Handle a known error

A matching except handles the conversion failure.

```
try: int("many")
except ValueError: print("invalid")
```

Readable input can still break a rule

Order form text		int(raw)		Positive count?
`"4"`	→	`4`	→	**Accepted**
`"0"`	→	`0`	→	**Rejected: zero**
`"many"`	→	`ValueError`		**No number to check**

```
count = int("0")
print(count, count > 0)
```

Conversion asks what number it is. Validation asks whether that number is allowed.

Syntax to read and reuse

Python / syntax	Meaning	When to use it
csv.DictReader(stream)	Read rows as dictionaries	CSV fields arrive as text
int(raw)	Convert numeric text	May raise ValueError
raise ValueError(...)	Reject an invalid value	Make failure explicit
try / except ValueError	Handle a known failure	Record a useful reason

Python Basics · sections 09, 11, 16 · use only when you need a lookup.

Clean and convert one string

```
raw = " 3 "
cleaned = raw.strip()
count = int(cleaned)
print(repr(raw))
print(count, type(count).__name__)
```

1. strip removes surrounding spaces; int converts text.

Handle a conversion failure

```
raw = "many"
try:
    count = int(raw)
    print("Converted:", count)
except ValueError:
    print("Not an integer")
print("Continue")
```

1. A failed int call jumps to except ValueError.

Check a missing field before string methods

```
row = {"source_id": "A"}
raw = row.get("count_text")
if raw is None:
    print("Missing count")
else:
    print(raw.strip())
```

1. get returns None when a key is absent.

Text repetition versus numeric multiplication

```
raw = "4"
print(raw, type(raw).__name__)
count = int(raw)
print(count, type(count).__name__)
print(raw * 2)
print(count * 2)
```

1. Expected: 4 str; 4 int; 44; 8.
2. Conversion produces a new value.
3. Try "many": locate the failing statement.

CSV fields arrive as strings

```
01 import csv
02 from io import StringIO
03 text = "batch_id,reported_containers\nDEMO,4\n"
04 row = next(csv.DictReader(StringIO(text)))
05 print(row["reported_containers"])
06 print(type(row["reported_containers"]).__name__)
07 assert row["reported_containers"] == "4"
```

1. The field contains text "4".
2. Its Python type is str.
3. Reading and converting are separate steps.

Numeric-looking CSV fields are still strings.

int(): convert numeric text

```
raw = "4"
containers = int(raw)
planned = 3 * containers
print(containers, planned)
assert planned == 12
```

1. Convert the string "4" to integer 4.
2. Use the converted value in multiplication.
3. Try invalid text and read the exception.

int() creates a number that arithmetic can use.

Conversion errors and validation errors

```
01 def parse_containers(raw):
02     count = int(raw)
03     if count <= 0:
04         raise ValueError("containers must be positive")
05     return count
06
07 for raw in ["4", "0", "many"]:
08     try:
09         print(parse_containers(raw))
10     except ValueError as error:
11         print("Review:", str(error))
```

1. 4 passes conversion and range checks
2. 0 fails the positive-request rule
3. many fails inside int

Zero can parse successfully and still be rejected.

argparse: read a typed argument

```
01  import argparse
02  parser = argparse.ArgumentParser()
03  parser.add_argument("--containers", type=int, default=4)
04  args = parser.parse_args(["--containers", "2"])
05  print(args.containers)
06  assert args.containers == 2
```

1. parse_args() returns an argument namespace.
2. type=int converts the supplied text.
3. args.containers retrieves the parsed value.

Argument parsing and range validation are separate steps.

Hands-on practice

Week_05.ipynb · Session 2

Python practice

P1–P2 · Core

Two small programs with approach hints.

P3 · Core

A complete program with normal and edge-case tests.

P4–P5 · Optional

An application and a harder challenge.

P1–P3: your code, expected results and an explanation. P4–P5 are optional.

Project studio

Proposal importer and audit

The project need

1

Need

Messages and table rows may contain missing or unclear counts.

2

Operation

Check each incoming row and keep a reason for its outcome.

3

Result

A human can follow up unclear information before it becomes an available offer.

Optional after core work: a supplied form calls the same tested function.

Keep source evidence through the pipeline

1

Read

Preserve source record ordinals and raw fields.

2

Normalize

Apply explicit transformations without inventing missing claims.

3

Decide

Validate individual rows and compare duplicate groups.

4

Record

Save a proposal or a reasoned hold/rejection.

A usable proposal and its audit answer different questions.

Project task

Supplied starting point

The CSV fixture and audit helper are provided. The helper supplies parsing, grouping and audit assembly.

You run / explain

Implement the row validator: normalize the ID and description, convert both count fields, and raise ValueError for invalid rows.

Check the evidence

Six-row audit → 2 proposed, 2 held, 2 rejected. Test nonnumeric, zero and negative counts; keep the original source evidence.

Account for every source row; proposals do not change availability.

Project checks

Step	What to show
Implementation	Implement validate_offer_row; run supplied audit helper.
Checks	Six rows → 2 proposed, 2 held, 2 rejected.

Account for every source row; proposals do not change availability.

Optional: a library provides reusable code

```
01 import math
02 answer = math.sqrt(9)
03 print(answer)  # 3.0
```

1. import makes a module available.
2. The dot selects its function.
3. A function call returns a value.

Optional: borrow the form, keep your own rule

You understand

Input counts and units
Arithmetic and if / else
Returned result and tests

Already supplied

Input validation
Notebook controls
Button wiring and display

Plain fallback

Edit ordinary Python inputs.
Call the same function.
Print and check the result.

No form is required for assessment. Missing or unresponsive? Use plain cells.

Further reading in English

1. ## Python CSV documentation

   https://docs.python.org/3/library/csv.html

2. ## Python errors and exceptions

   https://docs.python.org/3/tutorial/errors.html

Read one relevant section, then bring an example and a question.